Distributed Emotion Detection System

Major Project submitted to
Amity School of Engineering and Technology, Amity University
In partial fulfillment for the requirements for the degree of

I. [B. Tech + M. Tech] in Artificial Intelligence and Robotics
From
Department of Artificial Intelligence, Amity School of Engineering and Technology

By

Angshuman Basu (A910112520006)

Under the Guidance of
Prof. Pronaya Bhattacharya (309881)

Amity School of Engineering and Technology
Amity University
Kolkata – 700135
West Bengal, India
3rd December 2024

Amity School of Engineering and Technology

Amity University

Certificate

This is to certify that the project work entitled "Distributed Emotion Detection System" has been prepared by Angshuman Basu under my supervision and guidance. The thesis is his original work completed after careful research and investigation. The work of the thesis is of the standard expected of a candidate for I [B. Tech + M. Tech] (Artificial Intelligence and Robotics), and I recommend that it be sent for evaluation.

Date: 3rd December 2024

……...
Name and Signature of Guide

Forwarded By:

……...
Name and Signature of the
Head of Department

……...
Name and Signature of the
Head of Institute

Amity School of Engineering and Technology

Amity University

DECLARATION

I, **Mr. Angshuman Basu,** hereby declare that the project report entitled "**Distributed Emotion Detection System**" submitted to the Department of **AI, ASETK, AMITY UNIVERSITY KOLKATA** in partial fulfillment for the award of the Degree of **I [B.Tech + M.Tech] IN ARTIFICIAL INTELLIGENCE AND ROBOTICS** and that this Report has not previously formed the basis for the award of any other degree, Diploma, Associateship, Fellowship or other title.

Date: 3rd December 2024

……...
Signature of Student

DEDICATION

This project report is dedicated to researchers, technologists, and mental health professionals who strive to better understand and address the complexities of human emotions. It honors the efforts of those working to bridge the gap between artificial intelligence and human-centric applications, enabling systems that can recognize, validate, and respond to emotions in meaningful ways. Moreover, it is dedicated to individuals and communities who advocate for the ethical development of AI, ensuring that technological advancements benefit society at large by fostering empathy, understanding, and inclusivity.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to the academic community whose research and insights have greatly influenced my work. Special thanks to the online technical forums, including Stack Overflow and GitHub, for their invaluable support in resolving technical challenges.

I also appreciate my peers for their constructive feedback and encouragement throughout this journey.

I would also like to extend my thanks to the developers and contributors of open-source tools that were instrumental in the completion of this project.

Finally, I would like to extend my heartfelt gratitude to my professor, Mr. Pronaya Bhattacharya for his utmost valuable inputs and suggestions that have given my project broader horizons to aim at.

ANGSHUMAN BASU

A910112520006

ABSTRACT

This project introduces a distributed framework for emotion detection that aims to enhance the scalability, reliability, and interpretability of traditional emotion recognition systems. By leveraging a modular architecture, where each emotion is detected by an independent binary classification model, this approach alleviates the computational burden on a single model and ensures more accurate and interpretable predictions. The decentralized system uses a blockchain-inspired consensus mechanism, allowing for transparency, traceability, and validation of predictions across nodes, which strengthens the robustness of the model against potential failures and data inconsistencies.

In addition to addressing scalability and reliability, the framework integrates confidence-based decision-making, where both primary and secondary emotions are considered based on confidence scores. This nuanced emotional analysis helps overcome the limitations of conventional systems, which often output a single dominant emotion without accounting for secondary emotional states. The orchestrator in the system aggregates these predictions, providing a more comprehensive understanding of the detected emotions and making the system more adaptable to real-world applications.

This project holds the potential to significantly advance the field of emotional AI by improving its accuracy, interpretability, and reliability, particularly in high-stakes environments like mental health monitoring and human-computer interaction. Furthermore, the decentralized nature of the system makes it highly scalable, allowing for future expansion with minimal resource constraints. Overall, this approach opens new possibilities for more sensitive, context-aware, and efficient emotion detection, setting a new standard for AI-based emotional intelligence applications.

INTRODUCTION

The field of emotional AI has gained traction due to its applications in human-computer interaction, sentiment analysis, and mental health monitoring. Traditional emotion detection relies on centralized machine learning models for multiclass classification. However, these models face challenges with scalability, interpretability, and performance when handling complex emotional states. Such limitations necessitate modular, distributed solutions to overcome the shortcomings of centralized systems.

A critical gap in current research is the lack of confidence-based predictions and nuanced emotional insights. Existing models often predict a single dominant emotion, overlooking secondary emotions and confidence levels. This oversimplification is problematic in sensitive applications like therapeutic tools or conversational agents. Moreover, centralized models struggle with data inconsistencies, reducing their suitability for real-time, reliable decision-making.
This work proposes a distributed framework where each emotion is represented by an independent node equipped with a binary classification model. This modular setup improves scalability, reduces computational load, and enhances interpretability compared to multiclass models. A decentralized network architecture facilitates collaboration among nodes, yielding accurate and reliable predictions.

A blockchain-inspired consensus mechanism ensures transparency and robustness by validating predictions and maintaining a ledger of past decisions. An orchestrator aggregates predictions and ranks emotions based on confidence scores, providing nuanced emotional analysis. This approach addresses the limitations of existing models, combining decentralization, modularity, and consensus-driven validation.

The proposed framework offers a scalable, reliable, and interpretable solution for emotion recognition, setting a foundation for future innovations in emotional AI.

BACKGROUND STUDY/RELATED WORK

The advancement of emotion detection in AI has been significantly driven by research in areas such as natural language processing (NLP), deep learning, and machine learning. Emotion recognition plays a crucial role in understanding human interaction, enabling applications such as mental health monitoring, sentiment analysis, and human-computer interaction. Despite significant advancements, most emotion detection systems face limitations in terms of scalability, accuracy, and interpretability. This review discusses key components of the proposed project, each addressing specific research gaps to advance emotion recognition technology.

1. Emotion Detection with Natural Language Processing

Emotion detection from text has been widely studied within the field of NLP, with many models leveraging pre-trained transformers like BERT, RoBERTa, and GPT for text classification tasks. These models have shown remarkable success in recognizing emotions by fine-tuning them on emotion-labeled datasets, such as the EmoText dataset or the ISEAR corpus[1][2]. However, traditional models often struggle with scalability when tasked with handling multiple emotions simultaneously. Additionally, many systems rely on a single monolithic model to detect all emotions, which can lead to poor performance, as these models may become overwhelmed by the complexity of classifying diverse emotional states from text[3][4].

The proposed project addresses this gap by adopting a distributed system architecture where each emotion is classified by a dedicated binary classification model. This modular approach not only reduces the complexity of individual models but also allows for more focused and specialized emotion detection. This approach improves interpretability, as the system can provide explicit reasoning for each detected emotion, unlike conventional systems that output a single dominant emotion, which can be ambiguous and less insightful.

2. Confidence-Based Decision-Making

A major challenge in emotion detection systems is the confidence of predictions, particularly in situations where multiple emotions may coexist. Standard classifiers typically produce a single prediction without measuring how certain the model is about that prediction. For emotion detection, where context and nuances are critical, it is important to not only classify emotions but also quantify the model's confidence in its prediction. Existing systems do not always account for this uncertainty, which can lead to unreliable outputs, especially when emotions are mixed or subtle.

To overcome this limitation, the proposed system integrates confidence scores into the classification process. By utilizing softmax probabilities from the model outputs, the system provides a degree of certainty alongside each emotion prediction. Furthermore, the system considers secondary emotions with confidence scores, ensuring that a more complete emotional profile is generated. This feature not only enhances the accuracy of the system but also provides a richer understanding of emotional states, which is particularly valuable in complex human interactions[2][5].

3. Distributed and Decentralized Emotion Detection

Another critical limitation of current emotion detection systems is their reliance on centralized, monolithic architectures. These systems often face scalability issues when deployed in real-world applications, such as in large-scale sentiment analysis or human-computer interaction scenarios. Additionally, centralized systems are prone to single points of failure, making them less resilient and difficult to scale effectively[4][6].

The distributed approach adopted in this project leverages a decentralized system model, where each emotion detection task is handled by independent nodes. This approach ensures that each emotion has a dedicated node that specializes in detecting that specific emotion. By doing so, the system can scale more easily, handle increased loads efficiently, and minimize the risk of failure. Each node works independently, but the final prediction is aggregated by an orchestrator, which ensures that the system remains robust and fault-tolerant. This decentralized architecture draws

inspiration from the concepts of distributed computing, where tasks are divided among multiple processors to optimize performance[2].

4. Blockchain Consensus for Validation and Transparency

A growing challenge in machine learning and AI systems is the lack of transparency and accountability, particularly in decision-making processes. In critical applications such as healthcare or sentiment analysis for businesses, it is essential to ensure that the predictions made by AI systems are not only accurate but also transparent and verifiable. This is where blockchain technology can offer significant advantages. While blockchain is commonly associated with cryptocurrency, its core features—decentralization, immutability, and consensus—have been explored in AI for purposes such as secure data sharing and validating machine learning models[2][5].

The proposed project introduces blockchain-inspired mechanisms for validation and consensus across emotion detection nodes. By utilizing a consensus protocol, the system ensures that the emotion predictions are not only accurate but also transparent and auditable[4]. The ledger records each prediction, which can then be cross-verified, providing an additional layer of trustworthiness. This consensus-driven model helps reduce biases in the detection process and ensures that predictions are backed by a system that can be audited and verified, improving the overall reliability and accountability of the AI system.

5. Multi-Class Emotion Detection

Although the proposed system initially focuses on binary emotion classification for each model, the final goal is to extend this framework to handle multi-class classification. Most existing emotion detection systems treat emotions as separate, unrelated categories. This simplification fails to capture the complex interrelationships between emotions that humans experience in real life. For example, a person might feel both joy and surprise in response to an unexpected positive event, and these emotions can influence each other[1][6].

The system's modular architecture allows it to handle multi-class emotion detection by evaluating multiple emotions concurrently, ensuring that the system can detect complex emotional states. The orchestration of multiple emotion nodes and their aggregation provides a mechanism for the system to output a multi-emotion prediction, offering a more nuanced understanding of emotional states. By integrating confidence scores and the modular structure, the system is equipped to handle the overlap between emotions, ensuring that predictions are both accurate and comprehensive[4].

Rationale for the proposed solution:

The proposed solution addresses several key challenges in emotion detection systems by combining distributed architecture, confidence-based decision-making, and blockchain-inspired validation. Traditional emotion recognition systems often rely on monolithic models that struggle with scalability, accuracy, and interpretability, particularly in complex, multi-emotion scenarios. By decentralizing the task across specialized nodes for each emotion, the system not only reduces individual model complexity but also enhances overall scalability and fault tolerance. Incorporating confidence scores for primary and secondary emotions ensures that the predictions are more reliable, providing a richer understanding of emotional states. Additionally, leveraging blockchain mechanisms for validation and transparency introduces accountability, allowing for verifiable predictions and reducing biases in the process. This modular, transparent, and scalable approach is designed to overcome the limitations of existing systems and offer a more accurate, interpretable, and reliable solution for real-world emotion recognition applications.

METHODOLOGY

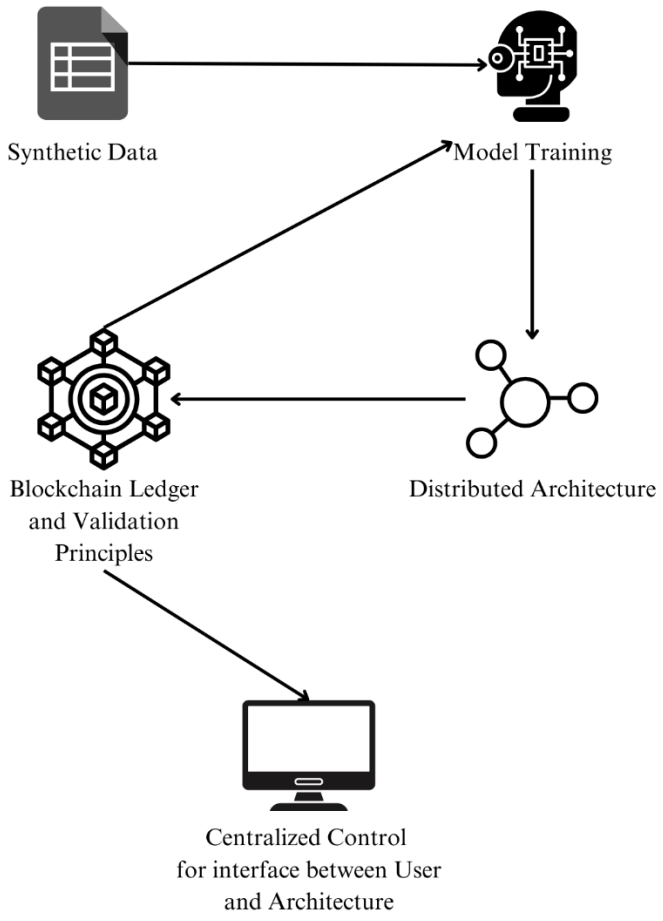

Fig 1: *The key processes involved in developing the architecture*

CHAPTER 1: MODEL TRAINING

The first critical step in the development of the emotion detection system involved training individual models for each of the five emotions: **Anger, Joy, Sadness, Surprise, and Fear.** Each emotion was treated as a binary classification task, where the model was tasked with determining whether the text conveyed the specific emotion (e.g., "Anger") or not. This approach allowed each model to specialize in detecting one emotion at a time, ensuring optimal performance for each case.

To train these models, we utilized **RoBERTa**, a transformer-based model pre-trained on a large corpus of text. RoBERTa is highly efficient for sequence classification tasks due to its ability to understand context in text, which is crucial for identifying emotions. RoBERTa was selected for its robustness and high performance in natural language processing tasks, making it an ideal candidate for this emotion detection task. The training process began with the preparation of a labeled dataset containing synthetic sentences associated with these five emotions. Each sentence was manually labeled as either indicating the target emotion or not, creating a balanced training set for each emotion.

The preprocessing pipeline involved several key steps:

1. **Text Tokenization: The input text was tokenized using the RoBERTa tokenizer**, which converts the text into token IDs that the model can process. This ensures the model receives the text in a format it can understand.

2. **Padding and Truncation:** Sentences were padded or truncated to a fixed length, ensuring consistent input sizes. This allowed the model to process text sequences efficiently without issues related to varying sentence lengths.

3. **Attention Masking:** An attention mask was applied to allow the model to focus on the relevant parts of the input text while ignoring padding tokens, improving the efficiency of the model's learning.

Once the preprocessing was completed, the model was fine-tuned on emotion-specific datasets for each emotion individually. This was a crucial step, as it ensured the model adapted well to the unique contexts of each emotion, rather than training a single multi-label model. The fine-tuning process adjusted the weights of the pre-trained RoBERTa model to better detect the emotional signals in the sentences. The model learned to distinguish the emotional context by examining the patterns of language used to express different emotions.

Model Evaluation and Validation

After training each model, we evaluated their performance using the key metric: **accuracy**. This metric provided insights into how well the models were performing in terms of both identifying the correct emotion and minimizing false positives and false negatives. The models were initially tested on a validation set, a portion of the dataset that was held out during training to ensure unbiased performance assessment.

Cross-validation was implemented to further validate the models. This technique involved splitting the dataset into multiple folds and training the model on different combinations of these folds while evaluating on the remaining fold. Cross-validation helped ensure the models' performance was consistent and not overfitting to a particular subset of the data.

Hyperparameter Tuning

To improve the model’s performance, **hyperparameter tuning** was conducted using grid search and random search. Key hyperparameters, including the **learning rate, batch size, number of epochs, and optimizer settings, were adjusted to find the most optimal configuration. Adam optimizer** was used with a learning rate scheduler to dynamically adjust the learning rate during training, which helped the model converge faster and avoid overfitting. **Early stopping** was employed to halt training if validation accuracy did not improve over several epochs, ensuring that the model would not continue to learn irrelevant patterns.

The dropout rate and batch normalization settings were also fine-tuned to prevent overfitting and improve the model's ability to generalize to new data. Through these iterative adjustments, the best combination of hyperparameters for each model was identified, ensuring the highest performance for detecting each emotion.

Final Model Selection

Once the models were trained and validated, the best-performing models for each emotion were selected based on their evaluation metrics, particularly accuracy and F1 score. A confidence score was also calculated. These models were then saved, along with their corresponding tokenizers, to prepare for deployment in a distributed system environment. The final models were tested on additional edge cases to ensure robustness, such as sentences that mixed emotions or ambiguous language, further ensuring the models' ability to generalize to diverse real-world data.

In summary, this chapter focused on training, evaluating, validating, and fine-tuning models for emotion detection. The use of **RoBERTa** and careful preprocessing, along with robust evaluation and hyperparameter optimization, ensured that the models were capable of accurately detecting emotions in text. These models formed the foundation for the subsequent chapters, where they would be deployed in a distributed system for real-time emotion detection.

CHAPTER 2: DISTRIBUTED SYSTEM ARCHITECTURE

The second stage of the project involved deploying the trained models in a distributed system environment to ensure scalability, reliability, and efficiency in detecting emotions in real-time. The deployment utilized Docker containers and the FastAPI framework to create lightweight, maintainable, and easily scalable services for each emotion detection model. This chapter elaborates on the framework setup, system architecture, and measures to achieve load balancing and fault tolerance.

Framework Setup and Containerization

To facilitate seamless deployment and scaling, each trained emotion detection model was encapsulated within its own Docker container. Docker provides an isolated environment that bundles the model, its dependencies, and the application code, ensuring consistency across different deployment environments.

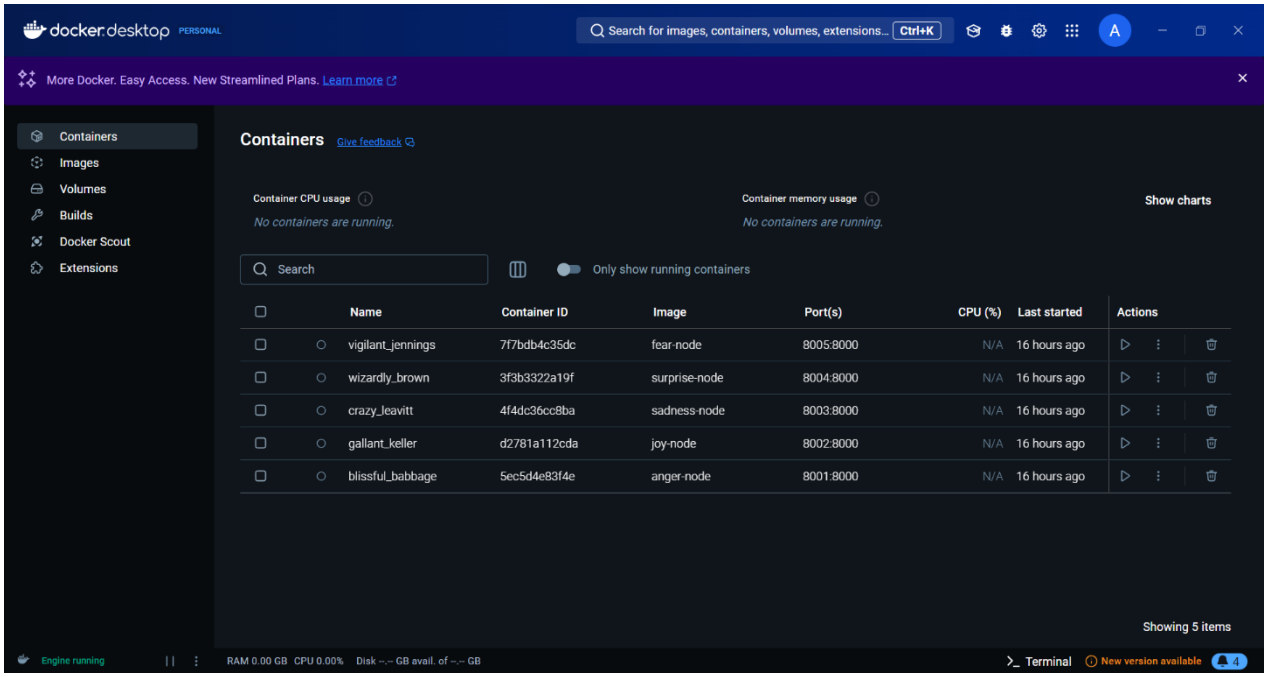

Fig 2: *The containers for all the emotion nodes*

Each Docker container included:

The Emotion Model and Tokenizer: Pre-trained models and corresponding tokenizers were integrated into the container, allowing each service to process requests independently.
FastAPI Endpoint: A FastAPI server was implemented in each container, exposing a /predict endpoint for receiving text input, processing it through the model, and returning the detected emotion with associated confidence scores.
Custom Configuration: Dockerfiles were tailored to include necessary libraries such as PyTorch, Transformers, and FastAPI. Port configurations were managed to ensure each service operated on a unique port within the system.
Using FastAPI, the application provided a lightweight and fast REST API interface for real-time interactions. FastAPI's asynchronous nature enabled efficient handling of multiple simultaneous requests, ensuring low latency.

System Architecture and Load Balancing

The distributed system architecture consisted of five nodes, each responsible for detecting one of the five emotions: Anger, Joy, Sadness, Surprise, and Fear. An orchestrator managed the communication between clients and these nodes.

To achieve load balancing, a round-robin approach was implemented using an orchestrator-level request scheduler. This approach ensured that incoming requests were distributed evenly across available nodes for each emotion, avoiding bottlenecks at any single node.

In addition to balancing the load, health checks were integrated into the orchestrator to monitor the availability and responsiveness of each node. Regular pings were sent to the /health endpoint of each node, allowing the orchestrator to detect and reroute traffic away from unresponsive or overloaded nodes.

Fault Tolerance and Redundancy

To enhance the system's reliability, fault tolerance mechanisms were implemented. Each emotion node was replicated multiple times across the distributed system, creating redundancy. If a primary node failed, the orchestrator could dynamically redirect requests to a backup node. This redundancy ensured uninterrupted service even in the event of hardware or network failures.

Furthermore, timeout and retry mechanisms were added to the orchestrator. If a node failed to respond within a predefined time frame, the orchestrator would retry the request or forward it to another node, ensuring a robust and fail-safe operation.

Scalability and Orchestration

The system was designed to scale horizontally, allowing new emotion nodes to be added seamlessly. This scalability was achieved through container orchestration tools like Docker Compose and Kubernetes, enabling dynamic scaling based on traffic demands.

Docker Compose: Initially used during development to manage multiple containers and their interconnections. It simplified the setup by defining configurations for all containers in a single YAML file.

Kubernetes: In a production environment, Kubernetes facilitated auto-scaling, load balancing, and health monitoring, making the system adaptable to varying workloads.

API Interaction and Testing

The FastAPI endpoints were rigorously tested to ensure accurate predictions and low response times. Unit tests were written for each node to validate the correct functioning of the /predict endpoint. Stress tests simulated high-traffic scenarios, evaluating the system's capacity to handle multiple requests simultaneously.

Each node returned a JSON response containing:

- **Primary Emotion Prediction**: The dominant emotion detected by the model.
- **Secondary Emotion Confidence**: The next most probable emotion and its confidence score.
- **Error Handling**: Detailed error messages in case of invalid inputs or processing failures.

CHAPTER 3: BLOCKCHAIN IMPLEMENTATION FOR PREDICTION VALIDATION

The integration of blockchain properties into the distributed emotion detection system introduces a robust mechanism for validating and auditing predictions. While security is not the primary focus in this implementation, blockchain technology is leveraged to ensure transparency, accountability, and the ability to validate the performance and consistency of the distributed system.

Framework Design and Rationale

Blockchain was incorporated into the system to maintain an immutable and decentralized ledger of all predictions made by the emotion detection nodes. Each prediction, including the detected emotion, confidence scores, and associated metadata, is stored as a transaction in the blockchain. By doing so, the system ensures that every prediction can be reviewed and validated at any point during or after deployment.

The choice of blockchain was motivated by its inherent properties of immutability and traceability, which align perfectly with the need for an auditable trail in emotion detection tasks. This ledger allows stakeholders to analyze historical predictions, identify inconsistencies, and validate model performance over time, thereby reinforcing confidence in the system.

Blockchain Structure and Components

The blockchain was implemented as a lightweight, distributed ledger tailored for recording prediction data. Its key components include:

Block Composition: Each block in the blockchain contains:

- **Prediction Data**: The primary and secondary predicted emotions along with their confidence scores.
- **Input Metadata**: Information about the input text, such as anonymized ID and preprocessing steps, ensuring reproducibility of the prediction.
- **Timestamp**: The exact time when the prediction was made.
- **Node Identifier**: The node that processed the input and generated the prediction.

Transaction Process:

Each prediction made by a node is immediately written as a transaction.
Once enough transactions are aggregated, they are bundled into a block.
The block is broadcasted across the system for validation before being appended to the chain.

Consensus Mechanism:

A lightweight consensus mechanism was implemented, ensuring that only valid blocks are appended to the chain.
The orchestrator acts as the coordinator, verifying the integrity of transactions and ensuring consistency across nodes.

Prediction Validation

One of the key benefits of the blockchain integration is its ability to validate predictions:

Model Performance Monitoring: By analyzing the historical data stored on the blockchain, discrepancies in predictions, such as consistently low confidence scores or high error rates for specific nodes, can be detected. These insights enable proactive isolation or retraining of underperforming models.

Traceability and Accountability: Each prediction is traceable to the specific node and time it was made. This facilitates post-hoc validation, particularly useful in applications like healthcare or customer service, where understanding the decision-making process is critical.

Consensus in Predictions: For inputs processed by multiple nodes (redundancy), the blockchain ledger records all predictions. This enables the system to identify consensus among nodes, further validating the accuracy and reliability of predictions.

Error Analysis: Misclassifications and edge cases can be reviewed by analyzing the chain. The ledger acts as a centralized repository for error data, helping developers refine models and improve overall system performance.

Implementation Workflow

The blockchain was integrated into the distributed system using the following workflow:

- **Prediction Logging**: After a node processes an input, its prediction data is encapsulated as a transaction.
- **Block Creation**: The orchestrator gathers transactions into blocks based on a predefined interval or transaction count threshold.
- **Block Validation and Appending**: Each block undergoes validation to ensure data integrity and is subsequently appended to the chain.
- **Access and Querying**: Authorized system components can query the blockchain to retrieve historical prediction data for analysis or debugging purposes.

Benefits of Blockchain Integration

The blockchain implementation enhances the emotion detection system in several ways:

- **Validation of Predictions**: The immutable ledger allows for consistent monitoring and validation of model outputs over time.

- **Performance Insights**: Analysis of blockchain data reveals patterns in predictions, enabling targeted improvements to specific models or nodes.
- **Transparency in Operations**: Stakeholders can audit predictions and ensure system reliability without directly interacting with the nodes or raw data.
- **Improved Reliability**: Historical data helps identify underperforming nodes, ensuring that the distributed system operates with optimal accuracy.

CHAPTER 4: ORCHESTRATOR DEVELOPMENT: INTERFACE FOR INPUT AND OUTPUT

The orchestrator serves as the pivotal interface between the distributed system and its users, streamlining input handling and output delivery. It acts as a mediator, coordinating communication between the client and the distributed nodes responsible for emotion classification. This chapter describes the design, functionality, and significance of the orchestrator in ensuring efficient and accurate interaction with the distributed emotion detection system.

Purpose and Role

The orchestrator's primary role is to provide a centralized interface through which clients can interact with the distributed system. It abstracts the complexities of the backend architecture, offering a seamless way to input text for emotion detection and retrieve predictions. By efficiently routing requests to the appropriate nodes and aggregating the outputs, the orchestrator simplifies the interaction process while maintaining high accuracy and responsiveness.

Design and Implementation

1. **Input Management:**
    a. The orchestrator accepts user inputs via a RESTful API endpoint implemented using the FastAPI framework.
    b. Inputs are preprocessed for consistency, including tokenization and ensuring compatibility with the format expected by individual nodes.

2. **Routing Requests:**
    a. Based on the user's input, the orchestrator determines which emotion(s) are relevant and routes the request to the corresponding distributed nodes.
    b. Each node processes a specific emotion, such as "Joy" or "Anger," and returns its prediction and confidence score.

3. **Output Aggregation**:
    a. The orchestrator collects responses from all queried nodes and compiles a comprehensive result.
    b. It determines the primary emotion based on the highest confidence score while considering secondary emotions within a confidence threshold.

4. **Fault Handling**:
    a. In the event of a node failure, the orchestrator dynamically reroutes requests to alternate nodes to ensure uninterrupted processing.
    b. A built-in failover mechanism ensures the system maintains reliability and consistency even under challenging conditions.

5. **Performance Tracking**:
    a. The orchestrator logs all inputs, outputs, and processing times, providing critical data for monitoring system performance and enabling debugging or optimization as needed.

Key Features:

1. **Input Abstraction**:
    a. Users interact with a single, intuitive API endpoint without needing to understand the distributed architecture.
    b. The orchestrator handles input preprocessing, such as ensuring text is tokenized and appropriately formatted for prediction.

2. **Dynamic Decision-Making**:
    a. The orchestrator intelligently decides which nodes to query, minimizing processing time while maximizing accuracy.
    b. For complex inputs involving multiple emotions, the orchestrator combines predictions from different nodes, creating a holistic output.

3. **Output Delivery**:
    a. The orchestrator consolidates predictions into a user-friendly format, including the detected emotion(s) and their confidence scores.
    b. It ensures that results are returned promptly, with the average response time optimized to handle concurrent requests efficiently.

4. **Scalability**:
    a. The modular design of the orchestrator allows for seamless integration of additional nodes as the system scales to handle new emotions or increased traffic.

5. **Error Recovery**:
    a. The orchestrator's failover capabilities minimize disruptions, ensuring that system failures do not compromise the user experience.

How the Orchestrator Interfaces Inputs and Outputs

The orchestrator is explicitly designed as the bridge between the client and the distributed nodes:

1. **User Input**:
    a. A client sends a text input to the orchestrator's API. For instance, "The movie was so touching, it made me cry."
    b. The orchestrator identifies the relevant emotions (e.g., "Sadness") and queries the appropriate node(s).

2. **Node Communication**:
    a. The input is routed to the corresponding nodes for processing. Each node returns its prediction (e.g., Sadness: Yes, Confidence: 0.92).

3. **Result Compilation**:
    a. The orchestrator aggregates all responses, prioritizes the emotion with the highest confidence, and checks for secondary emotions.

4. **Output Delivery**:

    The final output is delivered in a structured format:


```
{
"Primary Emotion": "Sadness",
"Confidence": 0.92,
"Secondary Emotions": [
{"Emotion": "Joy", "Confidence": 0.65}
]
}
```


Benefits of the Orchestrator as an Interface

1. **Simplified Interaction**:
    a. Users interact with a single endpoint, making the system accessible without requiring knowledge of the underlying architecture.
2. **Enhanced Usability**:
    a. The orchestrator provides well-organized results, ensuring that users receive meaningful insights from the system.
3. **Robustness**:
    a. The orchestrator's failover and fault-tolerant mechanisms ensure consistent performance and reliability.
4. **Modular Growth**:
    a. As additional emotions are added to the system, the orchestrator can adapt without significant changes to its interface or functionality.

The User Interface:

1. **Input Management:** The orchestrator accepts user inputs via a RESTful API endpoint, preprocesses them for consistency, and ensures compatibility with the format expected by the distributed nodes for emotion classification.
2. **Routing Requests:** Based on user input, the orchestrator intelligently routes requests to the appropriate distributed nodes, each responsible for processing a specific emotion, and retrieves the corresponding prediction and confidence score.
3. **Output Aggregation:** After collecting responses from the nodes, the orchestrator compiles a comprehensive result, determining the primary emotion based on the highest confidence and considering secondary emotions within a specified confidence threshold.
4. **Output Delivery:** The orchestrator consolidates the predictions into a user-friendly format, ensuring that the results, including the detected emotions and their confidence scores, are returned promptly to the client.

RESULTS AND DISCUSSION

The distributed emotion detection system achieved an overall accuracy of 0.96, demonstrating its effectiveness in identifying both primary and secondary emotions from text inputs. This high accuracy underscores the potential of the system for applications requiring nuanced emotional analysis, such as in customer service, healthcare, or social media monitoring.

Accuracy and Overlapping Emotions

One of the critical aspects of this project was the consideration of secondary emotions alongside primary emotions in accuracy calculations. Emotions often overlap due to their inherent relationships; for example, a text expressing "Joy" may also convey "Surprise," or a statement evoking "Sadness" could include traces of "Fear." By factoring secondary emotions into the evaluation process, the system captures these subtle connections, leading to a more comprehensive assessment of its performance.

However, the inclusion of secondary emotions introduced challenges. Cases of incorrect overlaps were observed, where the system detected unrelated or irrelevant secondary emotions. This indicates that while the current approach acknowledges the complexity of emotional expression, there is room for refinement in how overlapping emotions are predicted and validated.

Challenges and Limitations

1. **Data Constraints**:

    a. The system was trained and validated using smaller, manually curated datasets due to the limited availability of large-scale emotion-labeled data. While these datasets were sufficient for initial development, their size and scope restricted the ability of the models to generalize fully.

2. **Model-Emotion Relationships**:
    a. The architecture currently treats each emotion independently, without explicitly modeling relationships between emotions. This limitation can contribute to incorrect overlaps or missed associations between related emotions.

3. **Implications of Blockchain Validation**:

    a. The integration of blockchain technology provides an added layer of validation for predictions. By recording and verifying each prediction on an immutable ledger, the system ensures transparency and traceability. While this approach is robust, its effectiveness is tied to the quality of the underlying models. As larger and more diverse datasets become available, both individual model accuracy and the blockchain's validation mechanism are expected to improve significantly.

Summary

The distributed emotion detection system demonstrates strong initial performance with an accuracy of 0.96, highlighting its potential for practical applications. By considering secondary emotions, the system takes an essential step toward capturing the complexity of human emotional expression. However, limitations related to data scarcity and incorrect overlaps present areas for improvement. Future developments will focus on modeling emotion relationships, expanding the dataset, and refining blockchain validation to further enhance the system's accuracy and robustness.

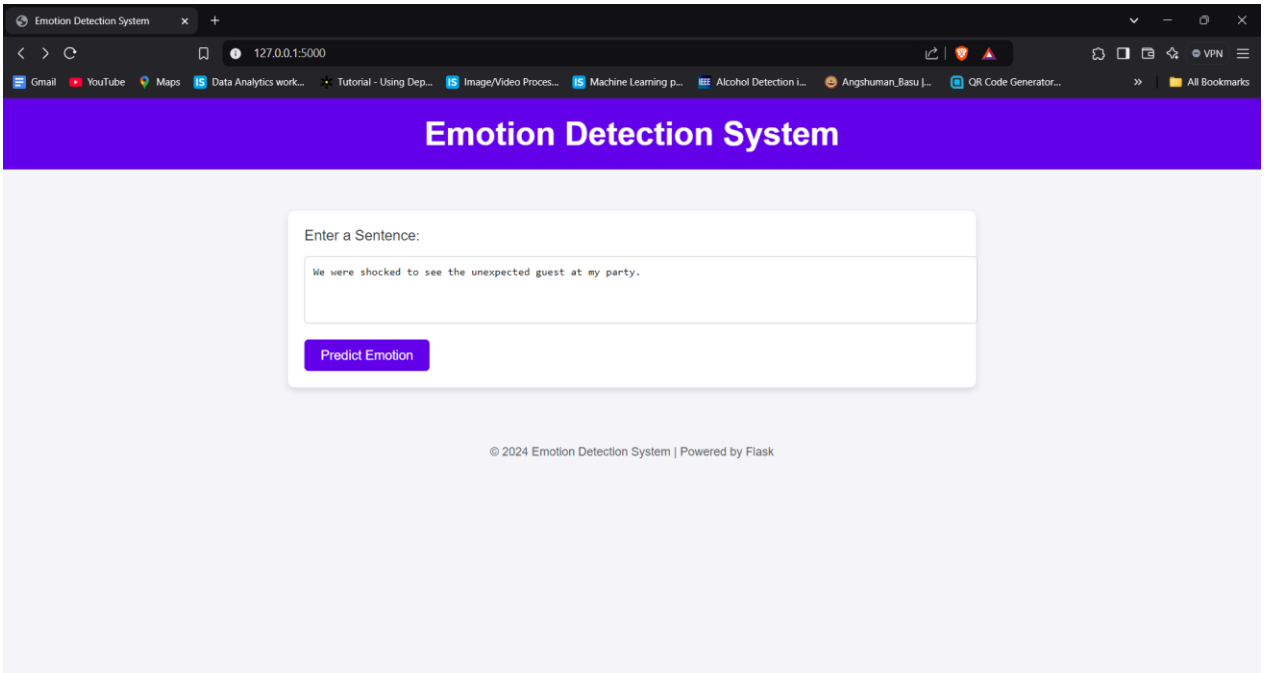

Fig 3: *A single sentence is accepted as input*

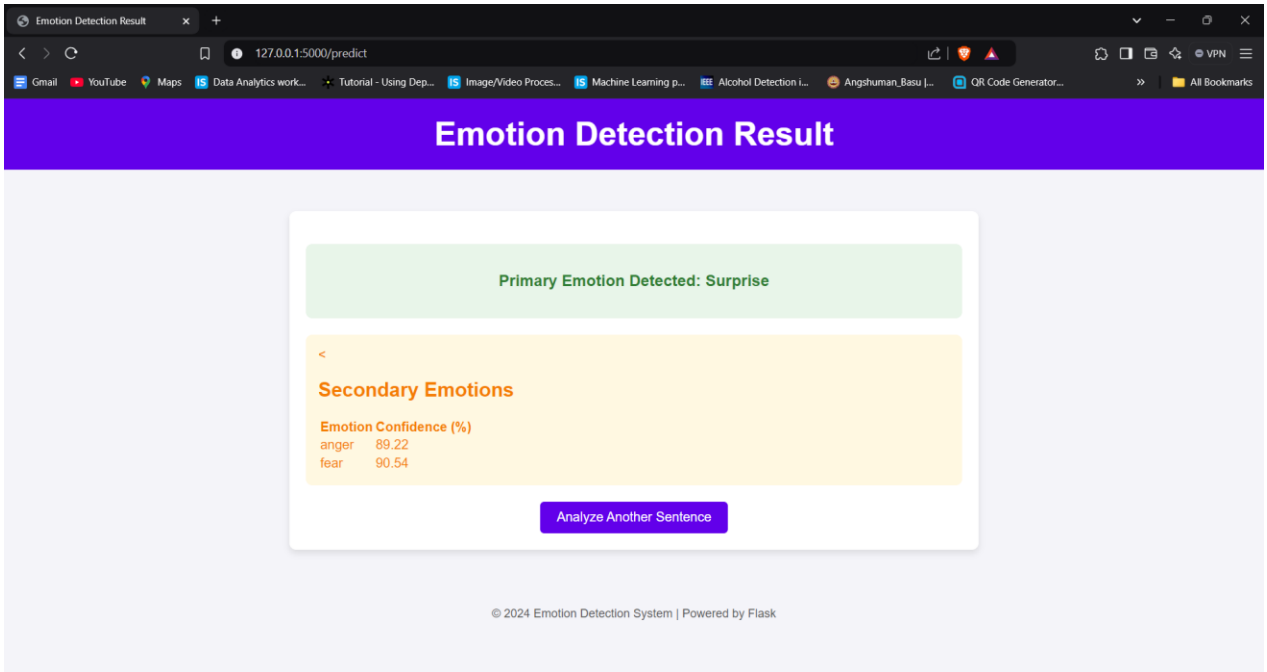

Fig 4: *The Primary emotion and Secondary emotions are displayed as output*

CONCLUSION AND FUTURE DIRECTIONS

The distributed emotion detection system developed in this project demonstrates a robust and innovative approach to understanding human emotions through text. With an accuracy of 0.96, the system effectively identifies primary and secondary emotions, acknowledging the nuanced overlap that often characterizes emotional expression. The integration of blockchain technology further ensures that predictions are transparent, traceable, and verifiable, offering a trustworthy framework for practical applications.

This project has the potential to significantly impact various fields. In customer service, it can enable businesses to better understand and respond to customer emotions, fostering stronger relationships and improved satisfaction. In healthcare, the system can aid in mental health monitoring by identifying distress or emotional changes in patient communications. Moreover, it could contribute to the broader field of human-computer interaction by enabling machines to respond empathetically, enhancing user experiences across multiple domains.

Despite its promising results, the project also highlights key challenges, including data scarcity and the need for more sophisticated modeling of emotional relationships. Addressing these issues will unlock even greater potential for this technology, making it more adaptable, inclusive, and accurate.

Future Directions

To build on the current success and realize its societal benefits, the following future directions are proposed:

1. **Modeling Emotion Relationships**:
    a. Future iterations will explicitly account for the interrelationships between emotions. This enhancement will reduce inaccuracies caused by incorrect overlaps and improve the system's ability to capture complex emotional nuances.

2. **Dataset Expansion**:
    a. Efforts will focus on gathering larger, diverse datasets representing a broader spectrum of emotions across various contexts and cultures. This will improve model generalization and ensure that the system remains equitable and effective for global applications.

3. **Enhanced Blockchain Validation**:
    a. The blockchain mechanism will be refined to incorporate more contextual information, such as timestamps and user feedback, further enhancing prediction reliability and enabling real-time auditing.

4. **Real-World Deployments**:
    a. Pilot programs will be initiated in sectors such as mental health, customer service, and social media monitoring to assess the system's impact and identify areas for optimization.

5. **Cross-Cultural and Multilingual Capabilities**:
    a. Recognizing that emotional expression varies across languages and cultures, future efforts will focus on making the system multilingual and culturally adaptive.

Societal Benefits

By bridging the gap between human emotional expression and machine understanding, this project has the potential to transform how technology interacts with society. It can contribute to mental health awareness, improve customer experiences, and foster a more empathetic digital ecosystem. By ensuring transparency and reliability through blockchain, the system also addresses ethical concerns, building trust in emotion-aware technologies.

This foundation paves the way for a future where machines are not only tools but empathetic collaborators in enhancing human well-being and fostering meaningful connections.

APPENDIX

Python Script for a single emotion (Joy Node)---

Training Model:

```
import torch
from torch.utils.data import DataLoader, TensorDataset
from transformers import RobertaTokenizer, RobertaForSequenceClassification, AdamW
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score
import pandas as pd
import numpy as np
import re
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from nltk.corpus import stopwords

def preprocess_text(text):
    text = re.sub(r'http\S+|www\S+|https\S+', '', text)  # Remove URLs
    text = re.sub(r'@\w+|#', '', text)  # Remove user mentions and hashtags
    text = re.sub(r'[^\w\s]', '', text)  # Remove punctuation
    tokens = word_tokenize(text.lower())
    lemmatizer = WordNetLemmatizer()
    tokens = [lemmatizer.lemmatize(word) for word in tokens if word not in
stopwords.words('english')]
    return " ".join(tokens)

import nltk
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('punkt')
nltk.download('wordnet')

file_path = "unique_synthetic_joy_dataset.csv"
df = pd.read_csv(file_path)
df['preprocess_text'] = df['Sentence'].apply(preprocess_text)

emotion = 'joy'  # Replace with the target emotion
df['label'] = df['Label'].apply(lambda x: 1 if x == 'Yes' else 0)

tokenizer = RobertaTokenizer.from_pretrained('roberta-base')
tokens = tokenizer(
    list(df['preprocess_text']),
    max_length=128,
```

```
    padding=True,
    truncation=True,
    return_tensors="pt"
)

# Prepare dataset
labels = torch.tensor(df['label'].values)
dataset = TensorDataset(tokens['input_ids'], tokens['attention_mask'], labels)
train_size = int(0.8 * len(dataset))
train_dataset, val_dataset = torch.utils.data.random_split(dataset, [train_size, len(dataset) -
train_size])

# Dataloaders
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=16)

# Model setup
model = RobertaForSequenceClassification.from_pretrained('roberta-base', num_labels=2)
optimizer = AdamW(model.parameters(), lr=5e-5)
criterion = torch.nn.CrossEntropyLoss()

# Training loop
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)

for epoch in range(9):  # Adjust number of epochs
    model.train()
    total_loss = 0
    for batch in train_loader:
        input_ids, attention_mask, labels = [b.to(device) for b in batch]
        optimizer.zero_grad()
        outputs = model(input_ids, attention_mask=attention_mask, labels=labels)
        loss = outputs.loss
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    print(f"Epoch {epoch + 1}, Loss: {total_loss / len(train_loader)}")

def compute_confidence(logits):
    probabilities = torch.softmax(logits, dim=-1)
    return probabilities[:, 1]  # Confidence for the positive class (emotion present)

# Evaluation with confidence
model.eval()
preds, true_labels, confidences = [], [], []
```

```
with torch.no_grad():
    for batch in val_loader:
        input_ids, attention_mask, labels = [b.to(device) for b in batch]
        outputs = model(input_ids, attention_mask=attention_mask)
        logits = outputs.logits
        confidence_scores = compute_confidence(logits)

        # Apply threshold for predictions
        threshold = 0.7  # Adjust as needed
        batch_preds = (confidence_scores > threshold).int().cpu().numpy()

        preds.extend(batch_preds)
        true_labels.extend(labels.cpu().numpy())
        confidences.extend(confidence_scores.cpu().numpy())

accuracy = accuracy_score(true_labels, preds)
f1 = f1_score(true_labels, preds)
print(f"Validation Accuracy: {accuracy}, F1 Score: {f1}")

# Confidence output example
for i in range(5):  # Print first 5 predictions
    print(f"True Label: {true_labels[i]}, Prediction: {preds[i]}, Confidence: {confidences[i]:.5f}")

# Saving the model
model_save_path = "joy_classification_model.pth"
tokenizer_save_path = "joy_tokenizer"

# Save model state dictionary
torch.save(model.state_dict(), model_save_path)

# Save tokenizer
tokenizer.save_pretrained(tokenizer_save_path)

print(f"Model saved to {model_save_path}")
print(f"Tokenizer saved to {tokenizer_save_path}")
```

Server code:

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import torch
from transformers import RobertaTokenizer, RobertaForSequenceClassification

# Initialize the FastAPI app
app = FastAPI()

# Paths to the saved files
m_path = r"joy_classification_model.pth"
t_path = "/app/tokenizer"

print(f"Tokenizer path: {t_path}")


# Load the model and tokenizer
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
tokenizer = RobertaTokenizer.from_pretrained(t_path)
model = RobertaForSequenceClassification.from_pretrained('roberta-base', num_labels=2)
model.load_state_dict(torch.load(m_path, map_location=device))
model.to(device)
model.eval()

# Request body format
class PredictionRequest(BaseModel):
    text: str

# Define prediction route
@app.post("/predict")
def predict_emotion(request: PredictionRequest):
    try:
        # Tokenize input text
        encoding = tokenizer.encode_plus(
            request.text,
            add_special_tokens=True,
            max_length=128,
            truncation=True,
            padding='max_length',
            return_tensors='pt'
        )

        # Move tensors to device
        input_ids = encoding['input_ids'].to(device)
        attention_mask = encoding['attention_mask'].to(device)
```

```
        # Get predictions
        with torch.no_grad():
            outputs = model(input_ids, attention_mask=attention_mask)
            logits = outputs.logits
            confidence_scores = torch.softmax(logits, dim=1)
            predicted_class = torch.argmax(logits, dim=1).item()
            confidence = confidence_scores[0, predicted_class].item()

        return {
            "prediction": "Joy" if predicted_class == 1 else "Not Joy",
            "confidence": confidence
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

Dockerfile:

```
# Base image
FROM python:3.8-slim

# Set the working directory in the container
WORKDIR /app

# Copy local files to the container
COPY joy_node.py .
COPY joy_classification_model.pth .
COPY joy_tokenizer /app/tokenizer

# Install dependencies
RUN pip install fastapi uvicorn transformers torch

# Expose port 8000
EXPOSE 8000

# Command to run the FastAPI server
CMD ["uvicorn", "joy_node:app", "--host", "0.0.0.0", "--port", "8000"]
```

Orchestrator:

```
from flask import Flask, request, jsonify, render_template
import requests

app = Flask(__name__)

# List of emotion nodes' URLs
emotion_nodes = {
    'anger': 'http://localhost:8001/predict',
    'joy': 'http://localhost:8002/predict',
    'sadness': 'http://localhost:8003/predict',
    'surprise': 'http://localhost:8004/predict',
    'fear': 'http://localhost:8005/predict'
}

# Confidence threshold to consider a secondary emotion
CONFIDENCE_THRESHOLD = 0.1

# Function to send request to the emotion node and get the response
def get_prediction(emotion, text):
    try:
        response = requests.post(emotion_nodes[emotion], json={'text': text})
        if response.status_code == 200:
            return response.json()
        else:
            raise Exception(f"Error in {emotion} node: {response.text}")
    except Exception as e:
        print(f"Error in {emotion} node: {e}")
        return {"prediction": "Not Emotion", "confidence": 0.0}

# Function to determine the primary and secondary emotions
def determine_emotions(predictions):
    # Get the primary emotion (highest confidence)
    primary_emotion = max(predictions, key=lambda x: x['confidence'])

    # Get secondary emotions that are close in confidence to the primary emotion
    secondary_emotions = {
        emotion['emotion']: round(emotion['confidence'] * 100, 2)  # Convert confidence to
percentage
        for emotion in predictions
        if abs(emotion['confidence'] - primary_emotion['confidence']) <=
CONFIDENCE_THRESHOLD
           and emotion != primary_emotion
    }
```

```
    return primary_emotion, secondary_emotions


# Route for the main interface
@app.route('/')
def home():
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    text = request.form['sentence']
    predictions = []

    # Get predictions from all emotion nodes
    for emotion in emotion_nodes.keys():
        prediction = get_prediction(emotion, text)
        predictions.append({
            'emotion': emotion,
            'confidence': prediction['confidence']
        })

    # Determine primary and secondary emotions
    primary_emotion, secondary_emotions = determine_emotions(predictions)

    # Render output page with results
    return render_template(
        'output.html',
        primary_emotion=primary_emotion['emotion'].capitalize(),
        secondary_emotions=secondary_emotions
    )


if __name__ == '__main__':
    app.run(debug=True)
```

Input Template:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Emotion Detection System</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f4f4f9;
            margin: 0;
            padding: 0;
        }
        header {
            background-color: #6200ea;
            color: white;
            padding: 20px 10px;
            text-align: center;
        }
        h1 {
            margin: 0;
            font-size: 2.5em;
        }
        .container {
            max-width: 800px;
            margin: 50px auto;
            background: white;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
        }
        label {
            font-size: 1.1em;
            color: #333;
        }
        textarea {
            width: 100%;
            padding: 10px;
            margin: 15px 0;
            font-size: 1em;
            border: 1px solid #ccc;
            border-radius: 5px;
            resize: none;
        }
```

```
        button {
            background-color: #6200ea;
            color: white;
            border: none;
            padding: 10px 20px;
            font-size: 1em;
            border-radius: 5px;
            cursor: pointer;
            transition: background-color 0.3s ease;
        }
        button:hover {
            background-color: #3700b3;
        }
        footer {
            text-align: center;
            padding: 20px;
            background-color: #f4f4f9;
            font-size: 0.9em;
            color: #555;
        }
    </style>
</head>
<body>
    <header>
        <h1>Emotion Detection System</h1>
    </header>
    <div class="container">
        <form action="/predict" method="post">
            <label for="sentence">Enter a Sentence:</label><br>
            <textarea id="sentence" name="sentence" rows="4" placeholder="Type your sentence
here..."></textarea><br>
            <button type="submit">Predict Emotion</button>
        </form>
    </div>
    <footer>
        &copy; 2024 Emotion Detection System | Powered by Flask
    </footer>
</body>
</html>
```

Output Template:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Emotion Detection Result</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f4f4f9;
            margin: 0;
            padding: 0;
        }
        header {
            background-color: #6200ea;
            color: white;
            padding: 20px 10px;
            text-align: center;
        }
        h1 {
            margin: 0;
            font-size: 2.5em;
        }
        .container {
            max-width: 800px;
            margin: 50px auto;
            background: white;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
        }
        .result-box {
            padding: 15px;
            margin: 20px 0;
            border-radius: 8px;
            background-color: #e8f5e9;
            color: #2e7d32;
            font-size: 1.2em;
            font-weight: bold;
            text-align: center;
        }
        .secondary-emotions {
            padding: 15px;
            margin: 20px 0;
```

```
            border-radius: 8px;
            background-color: #fff8e1;
            color: #f57c00;
            font-size: 1em;
        }
        button {
            background-color: #6200ea;
            color: white;
            border: none;
            padding: 10px 20px;
            font-size: 1em;
            border-radius: 5px;
            cursor: pointer;
            transition: background-color 0.3s ease;
            display: block;
            margin: 0 auto;
        }
        button:hover {
            background-color: #3700b3;
        }
        footer {
            text-align: center;
            padding: 20px;
            background-color: #f4f4f9;
            font-size: 0.9em;
            color: #555;
        }
    </style>
</head>
<body>
    <header>
        <h1>Emotion Detection Result</h1>
    </header>
    <div class="container">
        <div class="result-box">
            <p>Primary Emotion Detected: <strong>{{ primary_emotion }}</strong></p>
        </div>
        <div class="secondary-emotions">
            <<h2>Secondary Emotions</h2>
            <table>
                <thead>
                    <tr>
                        <th>Emotion</th>
                        <th>Confidence (%)</th>
                    </tr>
                </thead>
```

```
            <tbody>
                {% for emotion, confidence in secondary_emotions.items() %}
                <tr>
                    <td>{{ emotion }}</td>
                    <td>{{ confidence }}</td>
                </tr>
                {% endfor %}
            </tbody>
        </table>
    </div>
    <button onclick="window.location.href='/'">Analyze Another Sentence</button>
  </div>
  <footer>
    &copy; 2024 Emotion Detection System | Powered by Flask
  </footer>
</body>
</html>
```

REFERENCES

1. ArXiv Paper on Emotion Analysis in NLP: https://arxiv.org/html/2403.01222v1

2. Springer Article on NLP Applications: https://link.springer.com/article/10.1007/s10462-023-10509-0

3. Springer Book on Emotion Detection in NLP: https://link.springer.com/book/10.1007/978-3-031-72047-5

4. IEEE Xplore Document on Emotion Detection: https://ieeexplore.ieee.org/document/10072841

5. Viso.ai Overview of AI Emotion Recognition: https://viso.ai/deep-learning/visual-emotion-ai-recognition/

6. GitHub Repository on Emotion Detection in Text: https://github.com/SannketNikam/Emotion-Detection-in-Text